Designmodell

Modul: T4INF2003 – Software Engineering I
Kurs: TINF2024
Dozentin: Charlotte Chichlow
Semester: Frühjahr 2026

Projekt: Entwicklung eines Online Aufgaben-Planungssystems

Designmodell....1
1. Überblick und Zielsetzung....2
2. Systemübersicht und Architektur....2
    2.1 Produktbeschreibung....2
    2.2 Architekturentscheidung: Client-Server....2
    2.3 Schichten und Technologieentscheidungen....3
3. Statisches Modell....3
    3.1 Paketstruktur....3
    3.2 Klassen und Attribute....4
    3.3 Assoziationen und Vererbung....8
4. Dynamisches Modell....9
    4.1 Use-Case-Diagramm....9
    4.2 Sequenzdiagramm: Benutzer registrieren und anmelden....10
    4.3 Sequenzdiagramm: Aufgabe erstellen....11
    4.4 Sequenzdiagramm: Arbeitsplan generieren....11
        4.4.1 Beschreibung des Planungsalgorithmus....12
    4.5 Sequenzdiagramm: Aufgabe bearbeiten....15
    4.6 Identifizierte Methoden je Klasse....15
5. API-Spezifikation....16
6. Design Patterns....17
7. Frontend-Struktur....18
8. Scope der ersten Iteration (MVP)....18
9. Deployment und Entwicklungsinfrastruktur....19

1. Überblick und Zielsetzung

Das System „TeaPot" ist eine browserbasierte Webanwendung, die Einzelpersonen und Teams dabei unterstützt, Aufgaben nicht nur zu erfassen, sondern auch automatisiert und ressourcenbasiert zu planen. Im Gegensatz zu klassischen To-Do-Listen oder Kalender Applikationen überbrückt TeaPot die Lücke zwischen statischem Aufgabenmanagement und aktivem Kalendermanagement: Ein Planungsalgorithmus legt Aufgaben anhand von Priorität, Deadline, Abhängigkeiten und individuellem Arbeitsprofil selbstständig in freie Zeitfenster ein.

Die Designstudie hat folgende Ziele:

- Validierung der gewählten Technologien (React, C#, SQL Server)
- Prototypische Umsetzung zentraler Geschäftslogik
- Sammlung von Erfahrungen mit der geplanten Architektur
- Früherkennung von Risiken und Herausforderungen

2. Systemübersicht und Architektur

2.1 Produktbeschreibung

Das System richtet sich an Einzelpersonen und Teams, die Aufgaben strukturiert planen und koordinieren wollen. Benutzer können Aufgaben mit Prioritäten (Low, Medium, High), Deadlines, Zeitschätzungen und Abhängigkeiten konfigurieren sowie über ein persönliches Arbeitsprofil (tägliche Arbeitszeit, maximale Belastung) die Rahmenbedingungen für einen automatisch generierten Arbeitsplan definieren.

2.2 Architekturentscheidung: Client-Server

Das System setzt auf eine **drei-schichtige Client-Server-Architektur** mit klarer Trennung von Frontend (React), Backend (ASP.NET Core / C#) und Datenpersistenz (SQL Server). Die Kommunikation zwischen Frontend und Backend erfolgt ausschließlich über eine REST-API. Diese Architektur wurde gewählt, weil:

- **Separation of Concerns**: Frontend und Backend können unabhängig voneinander entwickelt, getestet und gewartet werden.
- **Skalierbarkeit**: Mehrere Clients können denselben Backend-Server nutzen, ohne dass Änderungen am Server notwendig werden.
- **Sicherheit**: Authentifizierung (JWT-Token) und Autorisierung werden zentral auf dem Server verwaltet.
- **Testbarkeit**: Backend-Logik und Frontend-Komponenten lassen sich isoliert testen.

2.3 Schichten und Technologieentscheidungen

Schicht	Technologie	Begründung
Frontend	React 18+	Wiederverwendbare Komponenten, Virtual DOM, breite Community
Backend	C# 11+ / ASP.NET Core 8+	Typsicherheit, async/await, Dependency Injection, LINQ
Datenbank	SQL Server 2019+	ACID-Compliance, EF-Core-Integration, Skalierbarkeit
API-Kommunikation	REST (JSON)	Standardisiert, gut unterstützt, einfach testbar
ORM	Entity Framework Core	Abstrahiert SQL, ermöglicht saubere Repository-Schicht
Authentifizierung	JWT (JSON Web Token)	Zustandslos, skalierbar, gut für SPAs geeignet

3. Statisches Modell

Hinweis zum Scope: In der ersten Iteration des Designmodells werden nur die für den MVP relevanten Kernobjekte (User, UserTask, WorkProfile, TimeInterval) detailliert modelliert und in den Sequenzdiagrammen verwendet. Die im Pflichtenheft definierten Objekte **Organisation**, **Arbeitsplan**, **Aufgabenblock** und deren Assoziationen sind fachlich zentral, werden aber für Iteration 1 nur im statischen Modell ergänzt, um die spätere Erweiterung zu ermöglichen. Im Fokus stehen zunächst die reinen Nutzer- und Planungsfunktionen.

3.1 Paketstruktur

Das System ist in folgende Pakete gegliedert:

- **Datamodels**: Enthält alle Entitäten (`User`, `UserTask`, `WorkProfile`, `TimeInterval`, `UserTaskDependency`, `Priority`)

- **ServiceLayer**: Unterteilt in drei Subsysteme:
  - **UserService**: Benutzerverwaltung (`IUserService`, `UserService`, `UserDto`)
  - **UserTaskService**: Aufgabenverwaltung (`IUserTaskService`, `UserTaskService`, `UserTaskDto`)
  - **PlanningEngine**: Planungslogik (`PlanningService`, `IUserTaskPlanner`, `UserTaskPlanner`, `SchedulingAlgorithm`, `WorkloadCalculator`)
- **Api**: REST-Endpunkte (`TaskController`, `PlanningController`, `AuthController`, `EndpointsExtensions`)
- **FrontendLayer – Components**: React-Komponenten (`GUI_Login`, `GUI_Mainpage`, `GUI_Overview`, `GUI_MyTeams`, `GUI_ProfileButton`, `GUI_Planner`, `GUI_Tasks`, `GUI_Settings`)

Die Trennung in diese Pakete folgt dem **Single-Responsibility-Prinzip**: Jedes Paket besitzt einen klar abgegrenzten Verantwortungsbereich. Die `PlanningEngine` ist als eigenständiges Subsystem gestaltet, um den Algorithmus künftig austauschen zu können (Strategy Pattern).

3.2 Klassen und Attribute

User

Repräsentiert einen registrierten Benutzer im System.

Attribut	Typ	Beschreibung
Id	Guid	Eindeutiger Bezeichner
UserName	string	Anzeigename
Email	string	Anmelde-E-Mail-Adresse
CreatedAt	DateTimeOffset	Erstellungszeitpunkt
UpdatedAt	DateTimeOffset	Letzter Änderungszeitpunkt
WorkProfile	WorkProfile	Navigation: verknüpftes Arbeitsprofil
UserTasks	ICollection<UserTasks>	Navigation: zugehörige Aufgaben

Designentscheidung: `Guid` statt `int` als Primärschlüssel wurde gewählt, um distributed-safe IDs zu ermöglichen und ID-Enumeration durch Angreifer zu verhindern.

UserTask

Kernentität für eine Aufgabe des Benutzers.

Attribut	Typ	Beschreibung
Id	Guid	Eindeutiger Bezeichner
TimeIntervals	List<TimeInterval>	Geplante Zeitfenster (vom Planer befüllt)
TimeEstimate	TimeSpan	Geschätzter Aufwand
Name	string	Bezeichnung der Aufgabe
Description	string	Detailbeschreibung
Deadline	DateTime?	Optionale Frist
Priority	Priority	Enum: Low / Medium / High
User	User	Navigation: zugehöriger Benutzer
Dependencies	ICollection<UserTask>	Navigation: Vorgänger-Aufgaben
Intensität	Intensität	Enum: LEICHT / NORMAL / INTENSIV, Zentrale Steuergröße für den Planungsalgorithmus

Designentscheidung: `Deadline` ist nullable (`DateTime?`), da nicht jede Aufgabe eine harte Frist besitzt. Die `TimeIntervals`-Liste wird nicht vom Benutzer, sondern von der `PlanningEngine` befüllt, um die Planung klar von der Eingabe zu trennen.

WorkProfile

Enthält die persönlichen Arbeitszeit-Einstellungen eines Benutzers.

Attribut	Typ	Beschreibung
Id	Guid	Eindeutiger Bezeichner
UserId	Guid	Fremdschlüssel zu User
WorkHours	List<TimeInterval>	Reguläre Arbeitszeiten
MaxDailyLoad	TimeSpan	Maximale tägliche Arbeitsbelastung

TimeInterval

Allgemeines Zeitfenster, das sowohl für Arbeitszeiten (WorkProfile) als auch für geplante Aufgaben-Slots (UserTask) verwendet wird.

Attribut	Typ	Beschreibung
startDate	DateTimeOffset	Beginn des Intervalls
endDate	DateTimeOffset	Ende des Intervalls

Designentscheidung: Ein generisches `TimeInterval` statt separater Klassen für Arbeitszeit und Planungs-Slot wurde gewählt, um Redundanz zu vermeiden und die Wiederverwendbarkeit zu erhöhen.

Organisation

Attribut	Typ	Beschreibung
Id	Guid	Eindeutiger Bezeichner
Name	string	Name der Organisation
Beschreibung	string	Optionale Beschreibung
VerfügbaresEinladungskontingent	int	Anzahl verfügbarer Einladungen
Memberships	ICollection<Membership>	Navigation: Zugehörige Mitglieder

Membership

Attribut	Typ	Beschreibung
UserId	Guid	Fremdschlüssel zu User
OrganizationId	Guid	Fremdschlüssel zu Organisation
Role	Role (Enum)	Rolle des Benutzers in der Organisation (NUTZENDE/ORGANISIERENDE)

Invitation

Attribut	Typ	Beschreibung
Id	Guid	Eindeutiger Bezeichner

Email	string	E-Mail der eingeladenen Person
InviteCode	string	Einmaliger Code zur Registrierung
OrganizationId	Guid	Fremdschlüssel zur einladenden Organisation

Aufgabenblock

Attribut	**Typ**	**Beschreibung**
Id	Guid	Eindeutiger Bezeichner
TaskId	Guid	Referenz auf die zugrundeliegende UserTask
ArbeitsplanId	Guid	Referenz auf den Arbeitsplan
StartDate	DateTimeOffset	Beginn des Planungs-Slots
EndDate	DateTimeOffset	Ende des Planungs-Slots
IsFixed	bool	Kennzeichnung, ob der Block vom Nutzer fixiert (angepinnt) wurde (vgl. 4.13)

Arbeitsplan

Attribut	**Typ**	**Beschreibung**
Id	Guid	Eindeutiger Bezeichner
UserId	Guid	Fremdschlüssel zu User
TaskBlocks	ICollection<Aufgabenblock>	Navigation: Zugehörige geplante Blöcke

UserTaskDependency

Modelliert eine gerichtete Abhängigkeit zwischen zwei Aufgaben (Aufgabe B kann erst beginnen, wenn Aufgabe A abgeschlossen ist).

Attribut	**Typ**	**Beschreibung**

TaskId	Guid	Nachfolger-Aufgabe
DependsOnTaskId	Guid	Vorgänger-Aufgabe

Designentscheidung: Die Abhängigkeit wird als eigene Entität (Assoziationsklasse) modelliert statt als einfache Liste im Task, da dies die Erweiterbarkeit (z. B. Abhängigkeitstypen) ermöglicht und ORM-seitig sauber abgebildet werden kann.

Priority (Enum)

Low | Medium | High

Intensität (Enum)
LEICHT | NORMAL | INTENSIV
Interfaces und Services

Klasse	**Typ**	**Verantwortung**
IUserService	Interface	Vertrag für Benutzerverwaltung
UserService	Klasse	Implementierung: CRUD für Benutzer, nimmt DbContext per DI
IUserTaskService	Interface	Vertrag für Aufgabenverwaltung
UserTaskService	Klasse	Implementierung: CRUD für Tasks, nimmt DbContext per DI
IUserTaskPlanner	Interface	Vertrag für den Planungsalgorithmus
UserTaskPlanner	Klasse	Implementierung: delegiert an SchedulingAlgorithm
SchedulingAlgorithm	Klasse	Kern Algorithmus: weist Aufgaben Time Intervals zu. Verwaltet intern einen Stack für Planungszustände zur Unterstützung des Backtracking

Designentscheidung: Alle Services implementieren Interfaces (`IUserService`, `IUserTaskService`, `IUserTaskPlanner`), um Dependency Injection und Unit-Testing mit Mock-Objekten zu ermöglichen (Inversion of Control).

3.3 Assoziationen und Vererbung

- `User` **1 – *** `UserTask`: Ein Benutzer besitzt beliebig viele Aufgaben; jede Aufgabe gehört genau einem Benutzer.

- `User` **1 – 1** `WorkProfile`: Jeder Benutzer hat genau ein Arbeitsprofil.
- `UserTask` * – * `UserTask` (via `UserTaskDependency`): Aufgaben können beliebig viele Vorgänger und Nachfolger haben.
- `WorkProfile` **1 – *** `TimeInterval`: Ein Arbeitsprofil definiert mehrere Arbeits-Zeitfenster.
- `UserTask` **1 – *** `TimeInterval`: Eine Aufgabe kann über mehrere geplante Zeitfenster verteilt sein.
- `UserTaskPlanner` implementiert `IUserTaskPlanner` (Realisierung).
- `UserService` implementiert `IUserService` (Realisierung).
- `UserTaskService` implementiert `IUserTaskService` (Realisierung).
- User **1 – *** `Membership`: Ein Benutzer hat mehrere Mitgliedschaften.
- Organisation **1 – *** `Membership`: Eine Organisation hat mehrere Mitglieder.
- `User` **1 – 1** `Arbeitsplan`: Jeder Benutzer hat genau einen Arbeitsplan.
- Arbeitsplan **1 – *** `Aufgabenblock`: Ein Arbeitsplan besteht aus mehreren Aufgabenblöcken.

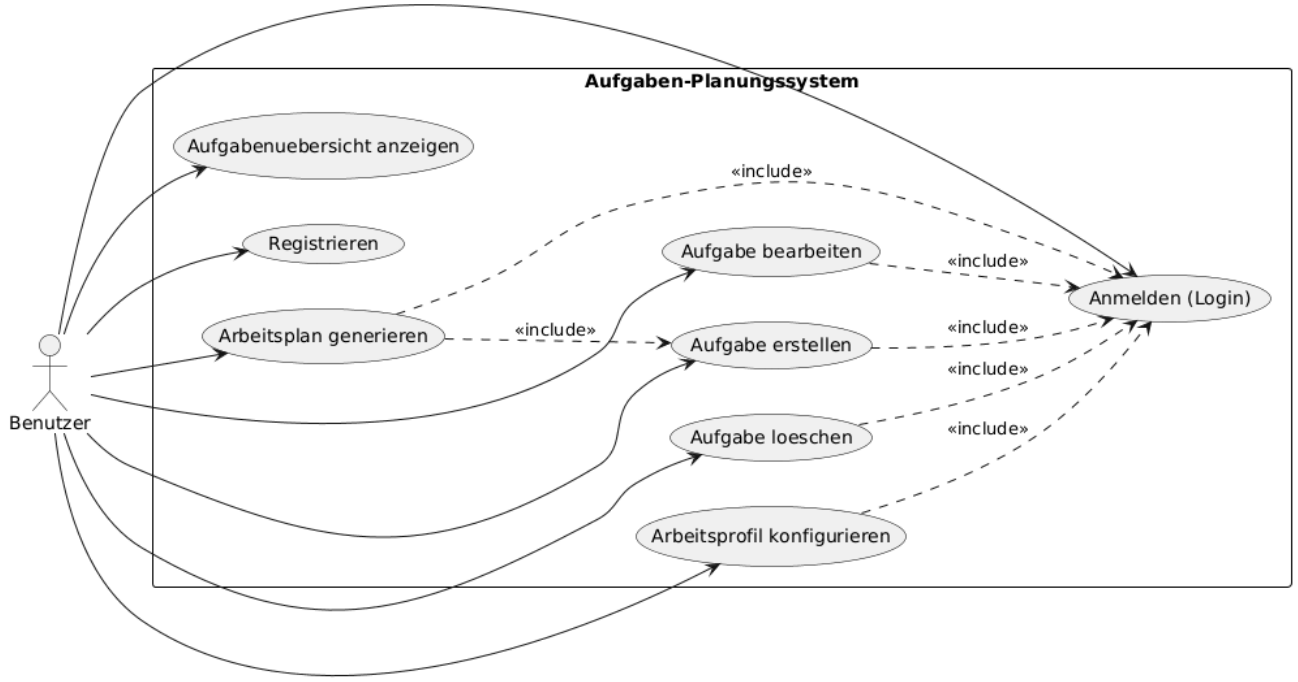
4. Dynamisches Modell

Das dynamische Modell beschreibt, wie die Klassen zur Laufzeit interagieren. Es wird anhand der vier wichtigsten Anwendungsfälle des MVP dargestellt.

4.1 Use-Case-Diagramm

Der einzige Akteur ist der registrierte Benutzer (nach Login). Der System-Rand umfasst das gesamte Aufgaben-Planungssystem.

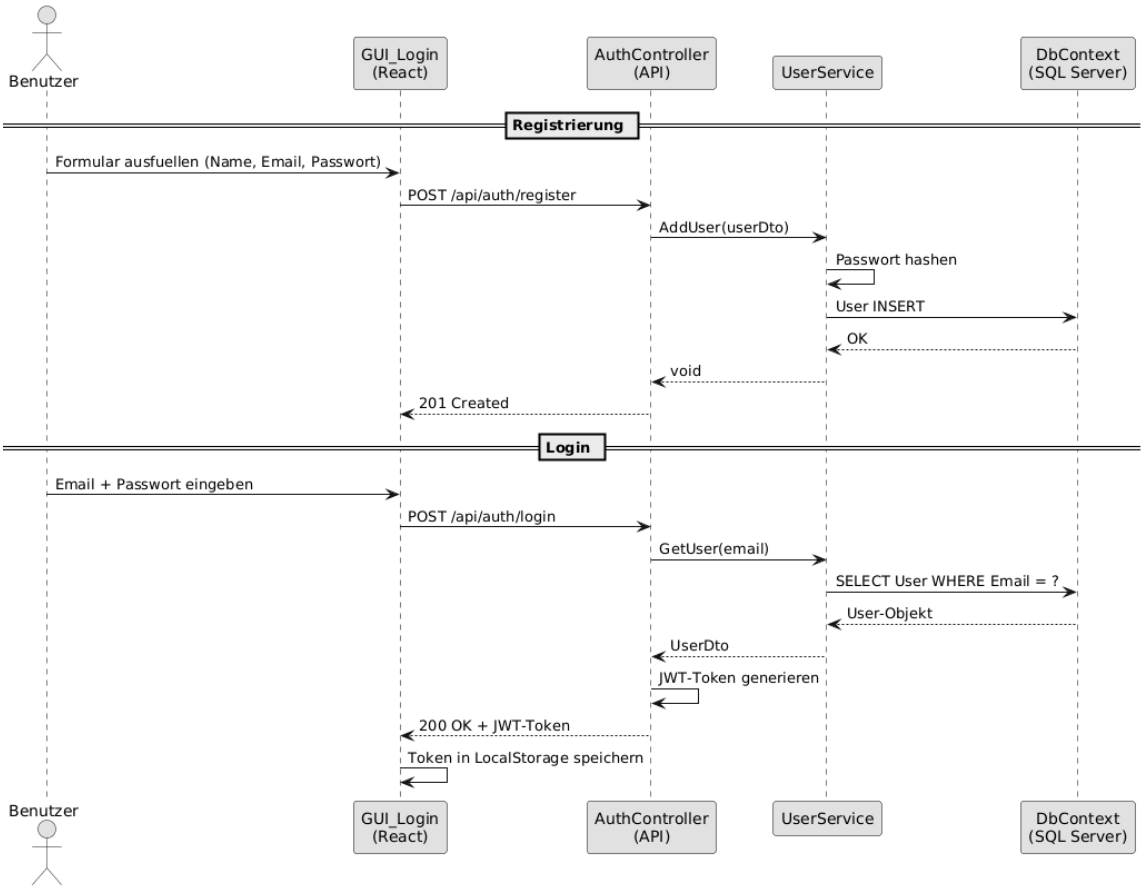
•**UC1:** Registrieren
•**UC2:** Anmelden (Login)
•**UC3:** Aufgabe erstellen
•**UC4:** Aufgabe bearbeiten
•**UC5:** Aufgabe löschen
•**UC6:** Arbeitsprofil konfigurieren
•**UC7:** Arbeitsplan generieren
•**UC8:** Aufgabenübersicht anzeigen



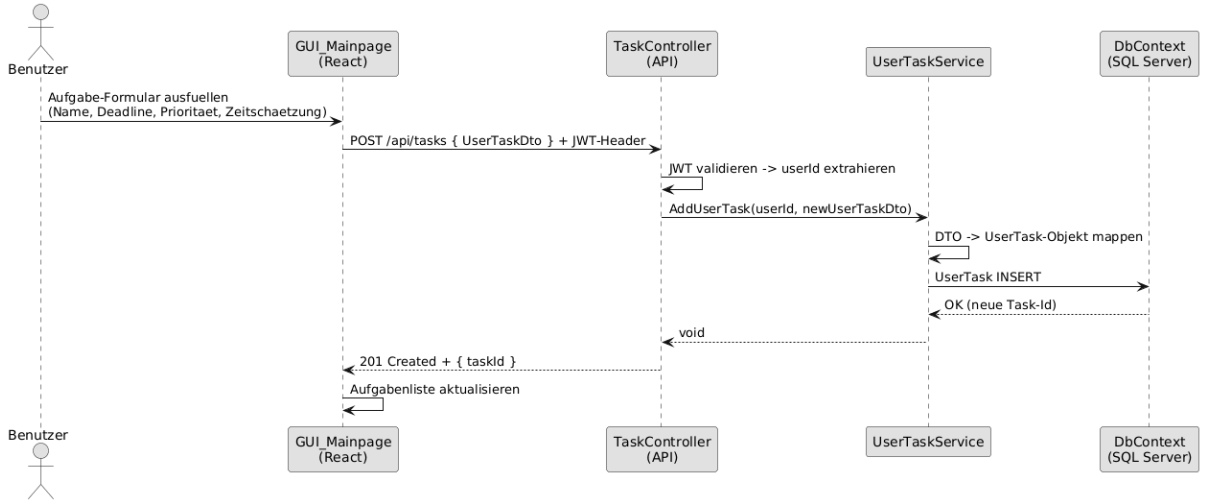
Begründung: UC7 (Arbeitsplan generieren) schließt UC3 ein, da ohne vorhandene Aufgaben kein Plan generiert werden kann. Alle Aktionen nach UC1 erfordern einen erfolgreichen Login (UC2).

4.2 Sequenzdiagramm: Benutzer registrieren und anmelden

Dieser Ablauf zeigt die Interaktion bei der Erstregistrierung und dem anschließenden Login.

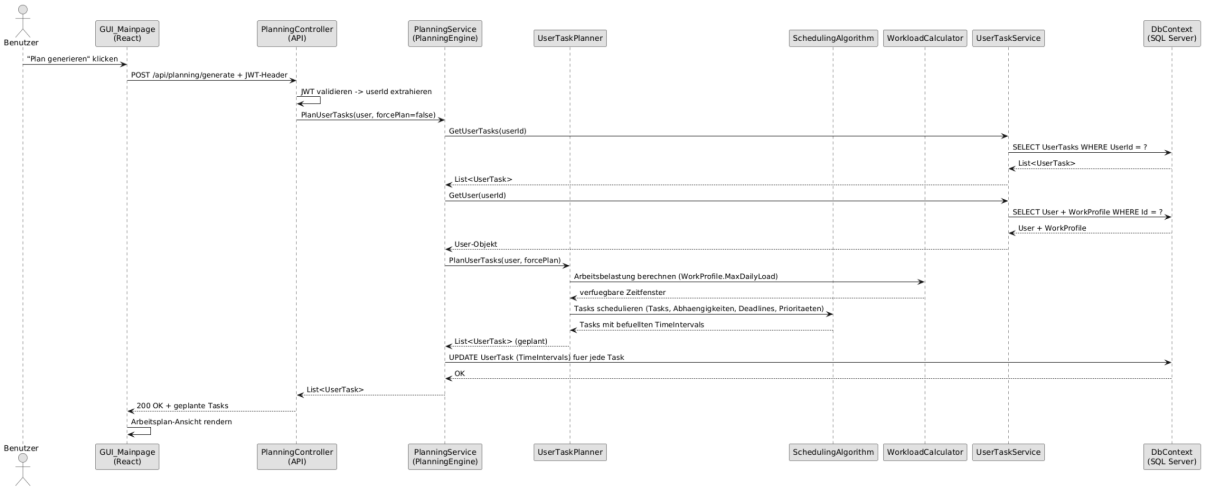


4.3 Sequenzdiagramm: Aufgabe erstellen



4.4 Sequenzdiagramm: Arbeitsplan generieren

Dies ist der komplexeste und für das System charakteristische Ablauf.



Begründung der Methoden:
Aus diesem Szenario ergeben sich folgende Methoden, die den Klassen zugewiesen werden:

- `PlanningController.GeneratePlan()` – delegiert an Service
- `PlanningService` (via `IUserTaskPlanner`) → `PlanUserTasks(user: User, forcePlan: bool): List<UserTask>`
- `SchedulingAlgorithm`: weist Aufgaben konkrete `TimeInterval`-Slots zu (Reihenfolge nach Priorität, Deadline, Abhängigkeiten)

4.4.1 Beschreibung des Planungsalgorithmus

Dieser Abschnitt beschreibt den Algorithmus zur automatischen Generierung eines Arbeitsplans (SchedulingAlgorithm). Die Darstellung fasst die wesentlichen Überlegungen strukturiert zusammen.

Ziele des Algorithmus:
- Aufgaben realistisch in den Tagesablauf einplanen.
- Individuelle Arbeitszeiten und Belastbarkeit der Nutzenden berücksichtigen.
- Pausen und feste Termine strikt respektieren.
- Zerstückelung des Tages in sehr viele kurze und psychologisch wenig sinnvolle Blöcke vermeiden.
- Kontrolliertes Backtracking nutzen, um alternative Planungsvarianten zu erproben.

1. Ausgangsdaten und Initialisierung

Der Algorithmus lädt zunächst alle benötigten Daten und legt den Planungszeitraum fest:
- Daten: Alle offenen Aufgaben (Restdauer, Deadline, Priorität, Intensität, …), das Arbeitsprofil der Nutzenden (Arbeitszeiten, Pausen, maximale tägliche Belastung) und feste Termine (Blockings).
- Initialisierung: Erzeugung von Arbeitszeitfenstern, Eintragen von Pausen/Terminen als Blockings zur Bestimmung freier Zeitslots.

- Steuerung: Initialisierung von Restdauern, Tagesbudgets, einem Stack für Planungszustände (für Backtracking) und einer Steuervariable (brauchtLeichteAufgabeDanach) zur Bevorzugung leichter Aufgaben nach intensiven Blöcken.

2. Auswahl der nächsten Aufgabe

Die Auswahl der nächsten Aufgabe erfolgt in einer Schleife und basiert auf folgenden Kriterien:

- Prüfung: Zuerst wird geprüft, ob das Flag brauchtLeichteAufgabeDanachgesetzt ist, um bevorzugt eine leichte Aufgabe (Intensität LEICHT) auszuwählen.
- Priorisierung: Falls keine leichte Aufgabe benötigt wird oder verfügbar ist, erfolgt die Auswahl nach: frühere Deadline, höhere Priorität, größere Restdauer.

3. Bestimmung des Kandidatenblocks und Behandlung kurzer Restdauern

Es wird eine Kandidatenblockdauer berechnet (min(Restdauer, maxBlockDauerMinuten)), um eine konfigurierte Obergrenze nicht zu überschreiten. Kurze Restdauern unterhalb der minimalen Blockdauer werden wie folgt behandelt:

- Anhängen: Die Restdauer wird an einen bereits existierenden Block derselben Aufgabe (am gleichen oder nächsten Tag) angehängt, wenn die maximale Blockdauer nicht überschritten wird.
- Restarbeit: Ist ein Anhängen nicht möglich, wird die Restdauer als „Restarbeit“ markiert und nicht als eigener Block geplant. Die Restdauer der Aufgabe wird auf 0 gesetzt.

4. Sicherung des Planungszustands (Backtracking-Basis)

Vor dem Einplanen wird der aktuelle Planungszustand auf einem Stack gesichert (bisher eingeplante Blöcke, Restdauern, Tagesbudgets), um bei späteren Konflikten ein kontrolliertes Backtracking zu ermöglichen.

5. Slot-Suche und dynamisches Splitting

Der Algorithmus sucht einen freien Zeitslot, der folgende Bedingungen erfüllt:

- Nicht durch Blockings belegt.
- Kombination aus Aufgabe, Block und Slot wurde noch nicht versucht.
- Der gesamte Kandidatenblock oder ein Teilblock (mindestens Mindestblockdauer) passt hinein.

Nach erfolgreicher Budgetprüfung (Einhaltung der maximalen Gesamt- und Intensivarbeitszeit) wird der Block eingeplant.

- Blockdauer: Gewählt wird das Minimum aus Kandidatdauer und Slotdauer, jedoch nicht unterhalb der Mindestblockdauer.
- Intensive Aufgaben: Bei als INTENSIV klassifizierten Blöcken wird brauchtLeichteAufgabeDanachauf true gesetzt.
- Dynamischer Split: Ist der Slot kleiner als die Kandidatdauer, aber größer als die Mindestblockdauer, wird ein Teil der Aufgabe verplant, und die Restdauer in späteren Schleifendurchläufen berücksichtigt.

Organisationskonfliktprüfung:

- Vor dem Einplanen eines Blocks wird geprüft, ob der Slot bereits durch Aufgabenblöcke desselben Nutzers aus anderen Organisationskontextenbelegt ist. Dies stellt die harte Randbedingung des Pflichtenhefts sicher, dass Aufgaben aus verschiedenen Organisationen sich zeitlich nicht überschneiden dürfen.

6. Lokales Backtracking bei Konflikten

Wenn Budgets verletzt werden oder kein Slot gefunden wird, erfolgt ein lokales Backtracking:

- Der zuletzt gesicherte Planungszustand wird wiederhergestellt, um die letzte Einplanung rückgängig zu machen.
- Die probierte Kombination wird als verwendet markiert, um Wiederholungen zu vermeiden.
- Falls die Obergrenzen (z. B. **MAX_VARIANTS_PER_BLOCK** und **MAX_BACKTRACK_STEPS**) für den Backtracking-Zähler überschritten werden, wird die Aufgabe als „aktuell nicht einplanbar“ markiert. Andernfalls wird eine alternative Entscheidung versucht (anderer Slot, anderer Tag, andere Aufgabenreihenfolge).

Dabei werden bereits getestete Kombinationen aus Aufgabe, Slot und Tag gespeichert, um Wiederholungen zu vermeiden, und es gelten definierte Obergrenzen für Backtracking-Schritte.

7. Behandlung von Restarbeiten am Ende

Nach der Hauptplanung werden markierte Restarbeiten (ursprünglich zu kurz für eigene Blöcke) gesammelt und am Tagesende in einem Sammelblock „Restarbeiten“ eingeplant, falls ein passender freier Slot gefunden wird. Ansonsten werden sie im Frontend lediglich als unstrukturierte Restarbeit kenntlich gemacht.

8. Globale Validierung und globales Backtracking

Vor dem Speichern wird eine globale Validierung durchgeführt (keine Überschneidungen, Einhaltung von Budgets und Blockdauern).

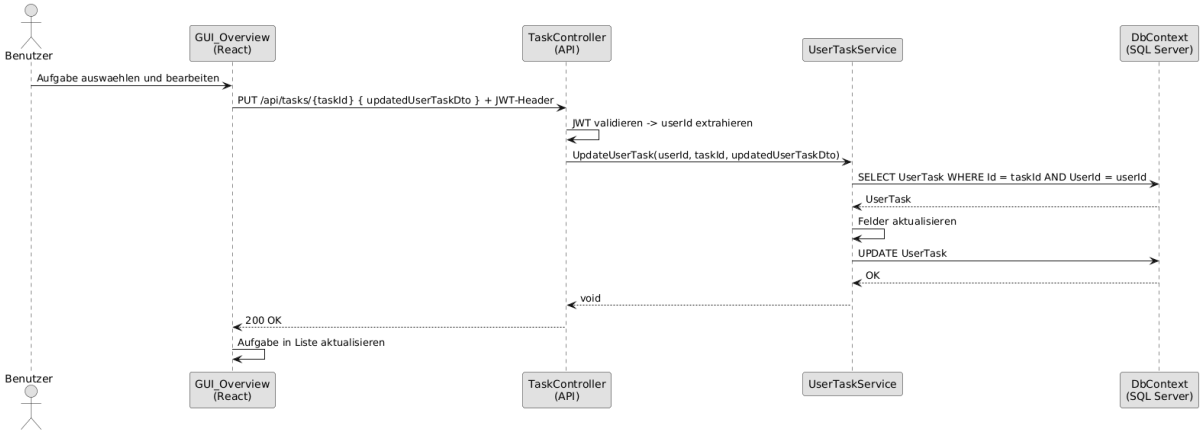
- Ungültiger Plan: Bei Ungültigkeit kann ein begrenztes globales Backtracking erfolgen, bei dem Planungsparameter moderat angepasst und ein alternativer Planungsdurchlauf gestartet wird.
- Abbruch: Führt auch dies zu keinem gültigen Plan, bricht der Algorithmus ab und informiert die Nutzenden über den Konflikt.

9. Zusammenfassung

Der beschriebene Algorithmus zur Arbeitsplangenerierung plant Aufgaben in sinnvoll dimensionierten Blöcken, begrenzt intensive Arbeitszeiten, steuert die Abfolge von intensiven und leichten Aufgaben, berücksichtigt Arbeitszeiten, Pausen und feste Termine, nutzt dynamisches Splitting, um flexibel auf freie Zeitslots zu reagieren, behandelt kurze Restdauern als Restarbeiten bzw. Puffer und nutzt lokales sowie globales Backtracking, um verschiedene Planungsvarianten zu erproben. Ziel ist die Erzeugung eines Arbeitsplans, der

sowohl die fachlichen Anforderungen als auch grundlegende Aspekte der kognitiven Belastung und praktischen Nutzbarkeit berücksichtigt.

4.5 Sequenzdiagramm: Aufgabe bearbeiten



4.6 Identifizierte Methoden je Klasse

Aus den Sequenzdiagrammen lassen sich folgende Methoden systematisch ableiten:

Klasse	**Methode**	**Auslöser**
`AuthController`	`LoginUser()`	Login-Sequenz
`AuthController`	`RegisterUser()`	Registrierungs-Sequenz
`TaskController`	`CreateTask()`	Aufgabe erstellen
`TaskController`	`GetTask()`	Aufgabenübersicht
`TaskController`	`UpdateTask()`	Aufgabe bearbeiten
`TaskController`	`DeleteTask()`	Aufgabe löschen
`PlanningController`	`GeneratePlan()`	Arbeitsplan generieren
`UserService`	`GetUser(userId)`	Login, Planung
`UserService`	`AddUser(userDto)`	Registrierung

`UserService`	`UpdateUser(userDto)`	Profil bearbeiten
`UserService`	`DeleteUser(userId)`	Konto löschen
`UserTaskService`	`GetUserTasks(userId)`	Aufgabenliste, Planung
`UserTaskService`	`AddUserTask(userId, dto)`	Aufgabe erstellen
`UserTaskService`	`UpdateUserTask(userId, taskId, dto)`	Aufgabe bearbeiten
`UserTaskService`	`UpdateUserTasks(userId, tasks)`	Plan speichern
`UserTaskService`	`DeleteUserTask(userId, taskId)`	Aufgabe löschen
`UserTaskPlanner`	`PlanUserTasks(user, forcePlan)`	Plan generieren
`SchedulingAlgorithm`	`GenerateSchedule(user)`	Aufgaben einplanen
WorkProfileService	`UpdateWorkProfile()`	Arbeitsprofil konfigurieren
PlanningService	`MoveTaskBlock()`	Arbeitsplan anpassen
OrganizationService	`RemoveMember()`	Nutzer aus Team löschen

5. API-Spezifikation

Methode	**Pfad**	**Auth**	**Input/Output DTOs**	**Status-Codes**
Get Org	organizations	OAuth	OrgId, OrgDTO	200, 400, 401, 404, 500
Get Orgs	organizations	OAuth	OrgIdList, OrgDTOList	200, 400, 401, 404, 500

Join Org	organizations/join	OAuth	InviteCode	200, 400, 401, 404, 500, 504
Leave Org	organization/leave	OAuth	OrgId	200, 400, 401, 404, 500, 504
CreateTask	tasks	OAuth	TaskDTO	201, 400, 404, 500,
GetTasks	tasks	OAuth	TaskIdList	200, 400, 401, 404, 500
DeleteTask	tasks	OAuth	TaskDTO	200, 400, 404, 500
UpdateTask	tasks	OAuth	TaskId, TaskDTO	200, 400, 404, 500
UpdateTasks	tasks	OAuth	TaskDTOList	200, 400, 401, 404, 500
ScheduleTasks	scheduler	OAuth	ScheduleDTO	200, 400, 401, 404, 500

6. Design Patterns

Das System setzt folgende Entwurfsmuster ein:

Pattern	**Anwendungsort**	**Nutzen**
Repository Pattern	`UserService`, `UserTaskService` greifen auf `DbContext` zu	Kapselt Datenbankzugriff, ermöglicht Austausch des Datenlayers
Dependency Injection	Alle Services erhalten `DbContext` und andere Abhängigkeiten per Konstruktor	Modularität, Testbarkeit (Mock-Objekte)

Service Layer Pattern	`UserService`, `UserTaskService`, `PlanningService`	Trennt Geschäftslogik von Controller und Datenschicht
Strategy Pattern	`IUserTaskPlanner` / `SchedulingAlgorithm`	Erlaubt Austausch des Planungsalgorithmus ohne API-Änderung
DTO Pattern	`UserDto`, `UserTaskDto`	Verhindert, dass Datenbankentitäten direkt exponiert werden

7. Frontend-Struktur

Die React-Anwendung gliedert sich in folgende Hauptkomponenten:

Komponente	Zuständigkeit
`GUI_Login`	Formular für Login und Registrierung
`GUI_Mainpage`	Navigations-Hub mit Buttons zu allen Unterbereichen
`GUI_Overview`	Aufgabenübersicht (Deadlines, anstehende Tasks, Wochenmengen)
`GUI_ProfileButton`	Profilbild, Name, Signed-In-Status
`GUI_MyTeams`	Liste aller Teams denen man zugehört
`GUI_Planner`	Kalenderübersicht der Aufgaben und Aufgabenverwaltung
`GUI_Tasks`	Listenübersicht der Aufgaben und Aufgabenverwaltung
`GUI_Settings`	Seite der Profileinstellungen

Der API-Zugriff erfolgt über einen zentralen **API-Service** im Frontend, der JWT-Token automatisch in den `Authorization`-Header einfügt. State wird per **React Context API** oder Redux Toolkit verwaltet (User State, Task State, Plan State).

8. Scope der ersten Iteration (MVP)

Funktion	Status	Begründung
F20 Registrierung & Login	**Go**	Basis Voraussetzung für alle weiteren Funktionen
F10 Aufgabe konfigurieren (CRUD)	**Go**	Kern Datengrundlage für die Planung
F50 Arbeitsprofil konfigurieren	**Go**	Pflicht Input für den Planungsalgorithmus
F40 Arbeitsplan generieren	**Go**	Kernfunktionalität des Produkts
F30 Organisierende einladen	**No-Go**	Komplexe Berechtigungen Logik
F60 Transparente Planung	**No-Go**	Erweiterte Visualisierung

Das Pflichtenheft beschreibt den vollständigen fachlichen Zielumfang. In der ersten Iteration dieses Designmodells werden nur die Kernfunktionen Registrierung, Aufgabenverwaltung, Arbeitsprofil und Arbeitsplangenerierung detailliert modelliert. Organisationsverwaltung und Einladungskontingente werden, wie in der Tabelle festgelegt, in späteren Iterationen erweitert.

9. Deployment und Entwicklungsinfrastruktur

Die Bereitstellung erfolgt via **Docker-Container** auf einem VPS oder Cloud-Anbieter (Azure/AWS). Die CI/CD-Pipeline (GitHub Actions) automatisiert Build, Test und Deployment:

1. Developer Push → GitHub Actions ausgelöst
2. Unit Tests (MSTest) + Integration Tests → müssen bestehen
3. UI Tests (Playwright) → müssen bestehen
4. Code-Coverage-Prüfung → Mindestwert 80%
5. Docker-Image bauen und in Registry pushen
6. Automatisches Deployment bei Merge in `main`

Versionierung erfolgt nach **Semantic Versioning** (MAJOR.MINOR.PATCH). Bei kritischen Fehlern nach Deployment kann auf das letzte stabile Docker-Image zurückgerollt werden.